galois_fields.py

Для использования скрипта для полей необходимо в блоке:

if __name__ == '__main__':

отредактировать значение переменных **'m'** и **'p'**, а затем - просто запустить сам скрипт.

По итогу в консоли будет выведена таблица:

```
File  Edit  Selection  View  Go  Run  ···                cryptography_on_python

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

PS C:\Users\Thunboo\Documents\GitHub\cryptography_on_python> & C:/Users/Thunboo/AppData/Local/Microsoft/WindowsApps/python3.10.exe c:/Users
 Посл.|      Многочлен        |  Степень  | Логарифм |
 0000 |          0            |     0     |    zoo   |
 0001 |          1            |     1     |    0     |
 0010 |          x            |     x     |    1     |
 0011 |        x + 1          |   x**4    |    4     |
 0100 |        x**2           |   x**2    |    2     |
 0101 |       x**2 + 1        |   x**8    |    8     |
 0110 |       x**2 + x        |   x**5    |    5     |
 0111 |     x**2 + x + 1      |   x**10   |   10     |
 1000 |        x**3           |   x**3    |    3     |
 1001 |       x**3 + 1        |   x**14   |   14     |
 1010 |       x**3 + x        |   x**9    |    9     |
 1011 |     x**3 + x + 1      |   x**7    |    7     |
 1100 |     x**3 + x**2       |   x**6    |    6     |
 1101 |   x**3 + x**2 + 1     |   x**13   |   13     |
 1110 |   x**3 + x**2 + x     |   x**11   |   11     |
 1111 | x**3 + x**2 + x + 1   |   x**12   |   12     |

-/\/\/\
```

elliptic_curves.py

Для использования скрипта необходимо изменить значение этих параметров:

```
2
3   #################
4   # Менять это:   #
5   #################
6                   #
7   MOD = 11        #
8   a, b = 3,4      #
9                   #
10  #################
```

Где MOD – модуль, по которому проводятся вычисления,

a, b – параметры эллиптической кривой.

По итогу в рабочей папке генерируется файл формата .json (вместо построения M x N таблицы, чтобы можно было каждую точку рассмотреть отдельно, а также упрощает вывод):

```
{} table.json > ...
1   {
2       "(0, 2)": {
3           "1": "(0, 2)",
4           "2": "(4, 6)",
5           "3": "(8, 1)",
6           "4": "(8, 10)",
7           "5": "(4, 5)",
8           "6": "(0, 9)",
9           "7": "O",
10          "8": "(0, 2)",
11          "9": "(4, 6)",
12          "10": "(8, 1)"
13      },
14      "(0, 9)": {
15          "1": "(0, 9)",
16          "2": "(4, 5)",
17          "3": "(8, 10)",
18          "4": "(8, 1)",
19          "5": "(4, 6)",
20          "6": "(0, 2)",
21          "7": "O",
22          "8": "(0, 9)",
23          "9": "(4, 5)",
24          "10": "(8, 10)"
25      },
26      "(4, 5)": {
27          "1": "(4, 5)",
```

Код программы приложен в приложении Б.

Приложение А

```
# Бройдо Глеб #
# ИБ-31       #
# 2025        #
###############

import sympy as sp
from sympy.abc import x

def remove_even_coeff_terms(expr):
    if not isinstance(expr, sp.Add):
        if expr.is_Number:
            coeff = expr
        else:
            coeff = expr.as_coeff_Mul()[0]

        if abs(coeff) % 2 == 0:
            return sp.Integer(0)
        else:
            return expr

    # Фильтруем слагаемые с нечетными коэффициентами
    filtered_terms = []
    for term in expr.args:
        # Получаем числовой коэффициент
        coeff = term.as_coeff_Mul()[0]
        # Проверяем четность
        if abs(coeff) % 2 != 0:
            filtered_terms.append(term / coeff)

    # Собираем новое выражение
    if not filtered_terms:
        return sp.Integer(0)
    elif len(filtered_terms) == 1:
        return filtered_terms[0]
    else:
        return sp.Add(*filtered_terms)

def make_tables(m: int, p: str):
    '''
    m - Длина последовательности
    p - Неприводимый многочлен из слагаемых типа:
    x\*\*n + x\*\*m + ... + 1 + 0
    '''
    zero_set = set('0')

    # Создаем словарь для x^i = Полином соответствущий последовательности
    # По минимуму для длины последовательности 2 нам надо 0, 1 и x, т.к.:
    ##############
    # 00 = 0     #
    # 01 = 1     #
    # 10 = x     #
    # 11 = x + 1 #
    ##############
    degrees = {
    0: 0,
    1: 1,
    x: x
    }
```

```
    # Затем мы добавляем все x^i, где i < m, так как они репрезентуют каждый из значащих битов последовательности
    for i in range(2, m):
        degrees[x**i] = x**i

    degrees[x**m] = 0
    x_m = p.split()[2:]
    for term in ''.join(x_m).split('+'):
        if term.startswith('x**'):
            degrees[x**m] += x**int((term.split('**')[1]))
        elif term.startswith('x'):
            degrees[x**m] += x
        else:
            try:
                degrees[x**m] += int(term)
            except:
                print(f'(ERROR):\nCan\'t add {term} to {degrees[x**m]} : {term} is not int!')

    for i in range(m+1, 2**m - 1):
        new_deg = sp.expand(degrees[x**(i-1)] * x)
        for j in range(i-1, 1, -1):
            new_deg = sp.expand(new_deg.subs(x**j, degrees[x**j]))
            new_deg = remove_even_coeff_terms(new_deg)
        degrees[x**i] = new_deg
    reverse_degrees = {value: key for key, value in degrees.items()}

    print(f" {'Посл.'}| {'Многочлен':^{m*5}} | {'Степень':^9} | {'Логарифм':^9} |")
    for i in range(2**m):
        seq = '0'*(2+m-len(bin(i))) + bin(i)[2:]
        polinom = sp.Add()
        for j in range(m):
            if int(seq[m-j-1]): polinom += x**j
        print(f" {seq:^{m}} |", end='')
        print(f" {'0' if set(seq) == zero_set else str(polinom):^{m*5}} |", end='')
        print(f" {str(reverse_degrees[polinom]):^9} |", end='')
        print(f" {str(sp.log(reverse_degrees[polinom])) if isinstance(reverse_degrees[polinom], int) else str(reverse_degrees[polinom].as_powers_dict()[x]):^9} |")

    # Разделяем таблицу с последовательностями и следующую
    sep = '/\\'
    print(f"\n-{sep*(23 + m*11)}-\n")

    # Заголовки для столбцов таблицы суммы
    for item in ['+', *degrees.keys()]:
        if item == '+': print(' ', end='')
        print(f"{str(item):^5}", end=' | ')
    print(f"\n{'-'*(48 + m*22)}")

    # Таблица суммы
    for degree_v in degrees:
        print(f"{str(degree_v):^5}", end=" || ")
        for degree_h in degrees:
            exp = degree_v + degree_h + x*0
            for j in range(2**m - 2, 1, -1):
                exp = sp.expand(exp.subs(x**j, degrees[x**j]))
                exp = remove_even_coeff_terms(exp)
            print(f"{str(reverse_degrees[exp]):^5}", end=" | ")
        print()

    # # Разделяем таблицу с суммы и следующую
    # sep = '/\\'
    # print(f"\n-{sep*(23 + m*11)}-\n")

    return

if __name__ == '__main__':
```

```python

    '''
    Для работы необходимо предоставить:

    m - Длина последовательности
    p - Неприводимый многочлен из слагаемых типа:
    x**n + x**(n-1) + ... + 1 + 0

    ! переменная 'p' задается строкой !
    '''

    m = 4
    p = 'x**4 + x + 1'

    make_tables(m, p)

```

Приложение Б

```
# Бройдо Глеб #
# ИБ-31       #
# 2025        #
#################
# Менять это:   #
#################
MOD = 11        #
a, b = 3,4      #
#################

import json

y2 = {}
for i in range(0, MOD):
    # print(f"{i}^2 = {i**2 % MOD}")
    if (i**2 % MOD) not in y2:
        y2[i**2 % MOD] = [i]
    else:
        y2[i**2 % MOD].append(i)
points = []
for i in range(0, MOD):
#     print(f"{i} <- x | y^2 = {(i**3 + i + 3 )% MOD} | \
# y2 = {y2[(i**3 + i + 3 )% MOD] if ((i**3 + i + 3 )% MOD) in y2 else None}")
    if not ((i**3 + a*i + b) % MOD) in y2:
        continue
    for y in y2[(i**3 + a*i + b) % MOD]:
        points.append( (i, y) )
print(points)

d = {}
for point in points:
    d[point] = {}
    d[point][1] = point
    # Calculating second point = 2*point_1
    '''            3*x**2 + a
        Lambda = ------------
                      2*y

        x' = Lambda**2 - 2*x
        y' = Lambda*(x - x') - y
    '''
    # If y==0 => infinite point 'O'
    if point[1] == 0:
        d[point][2] = "O"
    else:
        # Calculating Lambda:
        # Lambda = i:
        # 2 * y * Lambda = 3 * x**2 + a mod MOD
        for i in range(MOD):
            # print(f"{2*point[1] * i} % {MOD} = {2*point[1] * i % MOD} == {(3 * point[0]**2 + 1) % MOD} = {(3 * point[0]**2 + 1)} % {MOD}")
            if 2*point[1] * i % MOD == (3 * point[0]**2 + a) % MOD:
                l = i
                break
        x_ = l**2 - 2*point[0]
        x_ = x_ % MOD
        y_ = l*(point[0] - x_) - point[1]
        y_ = y_ % MOD
        d[point][2] = (x_, y_)

```

```python
    for k in range(3, MOD):
        if not isinstance(d[point][k-1][0], int):
            for K in range(k, MOD):
                d[point][K] = d[point][(K-1) % (k-1) + 1]
                # print(f"d[{point}][{K}] = d[{point}][{(K-1) % (k-1) + 1}] = {d[point][K]}")
            break

        if d[point][k-1][0] - point[0] == 0:
            d[point][k] = "O"
            continue

        # new point: point_1 + prev_point
        '''           y_2 - y_1
            Lambda = -----------
                      x_2 - x_1

            x_3 = Lambda**2 - x_1 - x_2
            y_3 = Lambda*(x_1 - x_3) - y_1
        '''

        # Calculating Lambda:
        # Lambda = i:
        # (x_2 - x_1) * Lambda = y_2 - y_1 mod MOD
        for i in range(MOD):
            if (d[point][k-1][0] - point[0]) * i % MOD == (d[point][k-1][1] - point[1]) % MOD:
                l = i
                break
        x3 = l**2 - point[0] - d[point][k-1][0]
        x3 = x3 % MOD
        y3 = l * (point[0] - x3) - point[1]
        y3 = y3 % MOD
        print(f"\
\npoint:{point}\
\nk={ }\
\np1 + prev = {point} + {d[point][k-1]}\
\nLambda = {l} | {(d[point][k-1][0] - point[0])} * i % {MOD} == {(d[point][k-1][1] - point[1])} % {MOD}\
\nx3, y3 = {l**2 - point[0] - d[point][k-1][0]}, {l * (point[0] - x3) - point[1]}\
\nx3, y3 = {x3}, {y3}")
        d[point][k] = (x3, y3)

def convert_tuples_to_strings(obj):
    if isinstance(obj, dict):
        return {str(k): convert_tuples_to_strings(v) for k, v in obj.items()}
    elif isinstance(obj, tuple):
        return str(obj)
    elif isinstance(obj, list):
        return [convert_tuples_to_strings(item) for item in obj]
    else:
        return obj

converted_data = convert_tuples_to_strings(d)
json_output = json.dumps(converted_data, indent=2)
# print(json_output)

with open("table.json", 'w') as f:
    json.dump(converted_data, f, indent=4)
```